

CrypticSync

Employee Attendance Tracking System

crypticsync.rakshitr.co.in

Prepared by	Rakshit R
Stack	Angular 17 · ASP.NET Core 8 · MySQL 8.0
Hosting	Vercel + Cloudflare Tunnel + Docker
Version	1.0 — April 2026

Table of Contents

Table of Contents	2
1. Executive Summary	4
2. Assumptions	5
3. Technology Stack & Justification.....	6
3.1 Chosen Stack	6
3.2 FastAPI vs ASP.NET Core	6
3.3 Hosting Architecture	7
4. Feature List.....	8
4.1 What They Asked For.....	8
Authentication & Access Control	8
Individual Employee Dashboard	8
Manager Dashboard (Hierarchical)	8
Data Source Integration	8
4.2 What We Give Extra (Value-Adds)	8
4.3 Future Scope.....	9
5. Diagrams	10
5.1 Use Case Diagram	10
5.2 System Architecture Diagram	11
5.3 Data Flow Diagram.....	12
5.4 Sequence Diagram.....	13
5.5 Entity Relationship Diagram	14
5.6 Deployment Diagram.....	15
6. Database Schema	16
6.1 Table: employees	16
6.2 Table: users	16
6.3 Table: departments	17
6.4 Table: reporting_hierarchy.....	17
6.5 Table: attendance_daily	17
6.6 Table: attendance_monthly	18
6.7 Table: xls_upload_log	19
7. Stored Procedures	20
7.1 sp_SyncDailyAttendance.....	20
7.2 sp_GetTeamAttendanceHierarchy	20
7.3 sp_GetMonthlyAttendanceSummary	21
7.4 sp_GetEmployeePunchTimeline	21
8. Angular Components & Pages	22
8.1 Application Structure	22

9. API Endpoints (ASP.NET Core).....	25
9.1 Authentication	25
9.2 Attendance	25
9.3 Employees	25
9.4 Users & Hierarchy	26
9.5 Sync & Upload	26
10. COSEC API Integration Plan	27
10.1 API Overview	27
10.2 Key Field Mapping — Daily Attendance	27
10.3 Demo / Mock Strategy	27
11. Security Approach.....	29
12. Alternative Approach — When API is Unavailable	30
12.1 Why the API May Be Unavailable	30
12.2 The Fallback: XLS Upload Pipeline	30
12.3 XLS Upload Flow.....	30
12.4 Alternatives Considered and Rejected.....	31
13. Future Scope	32
Phase 2 — Leave Management	32
Phase 2 — Notifications & Alerts.....	32
Phase 2 — Advanced Sync.....	32
Phase 3 — Analytics	32
Phase 3 — Integrations & Accessibility	32
Appendix A — Docker Compose Structure	33
Appendix B — DNS Configuration	33
Appendix C — COSEC Status Code Reference	33

1. Executive Summary

The organization uses a Matrix COSEC biometric system to track employee attendance. While the system accurately records punch-in and punch-out events, it provides no employee-facing or manager-facing dashboard. Employees cannot view their own attendance records, and managers have no visibility into their team's attendance status. The only available data export is a formatted XLS file, and the system exposes a REST API (Matrix COSEC Web API V2.0) for programmatic data access.

CrypticSync bridges this gap. It is a secure, role-based web application that pulls attendance data from the COSEC APIs, stores it in a MySQL database, and presents it through a responsive Angular frontend. Employees can view their own attendance history from their date of joining. Managers can view their own attendance as well as the attendance of their entire reporting hierarchy, including sub-teams. HR/SuperUsers have full administrative access including user management, hierarchy configuration, and data synchronization.

The application is hosted at crypticsync.rakshitr.co.in (Angular on Vercel) with the backend API at api.rakshitr.co.in/crypticsync (ASP.NET Core 8 on Docker, exposed via Cloudflare Tunnel).

2. Assumptions

- The Matrix COSEC biometric system is the source of truth for all attendance data. CrypticSync is a read-and-display layer — it does not write data back to COSEC.
- The COSEC API (<http://<servername>/api.svc/v2/...>) is typically on a private network. For demonstration purposes, COSEC API responses are mocked in the backend. In production, the COSEC server URL and credentials would be configured via environment variables.
- The reporting hierarchy (who reports to whom) is not stored in the COSEC system and will be maintained manually in CrypticSync by the HR/SuperUser.
- User accounts are created manually by HR/SuperUser. There is no self-registration. Login credential is Employee ID + Password.
- Historical attendance data is pre-synced from each employee's Date of Joining. The system will store all available historical data on initial setup.
- All timestamps are in IST (Indian Standard Time, UTC+5:30).
- English is the only supported language for Phase 1. Localization and ADA accessibility are planned for future phases.
- The COSEC system exports XLS files in the format of the provided sample — company header rows, per-employee sections, and columns: Sr No, Date, Shift, First IN, Last OUT, 1st Half, 2nd Half, Gross Work Hrs, N-Punch Work Hrs, Man Entry.
- Passwords are hashed using BCrypt before storage. Plain-text passwords are never stored.
- Docker and Docker Compose are used to manage the backend containers on the local PC server.

3. Technology Stack & Justification

3.1 Chosen Stack

Layer	Technology	Reason
Frontend	Angular 17+	Component-based architecture, strong TypeScript support, Angular Material UI library, built-in route guards for role-based access
Backend API	ASP.NET Core 8 (C#)	Professional experience with C#, excellent MySQL/Dapper integration, battle-tested JWT auth, strong tooling (Visual Studio)
ORM / DB Access	Dapper (micro-ORM)	Lightweight, ideal for Stored Procedure calls, maps result sets to C# models cleanly, no EF overhead
Database	MySQL 8.0	Open source, excellent Stored Procedure support, widely supported in Docker, sufficient for this workload
Auth	JWT Bearer Tokens	Stateless, scalable, pairs cleanly with Angular HTTP Interceptors for attaching tokens to requests
Frontend Hosting	Vercel	Free tier, auto-deploy from GitHub, global CDN, auto-SSL via Let's Encrypt
API Exposure	Cloudflare Tunnel	No static IP needed, free, enterprise-grade security, auto-SSL, exposes local Docker host to internet
Containerization	Docker + Docker Compose	Reproducible environment, clean separation of nginx/api/db, easy restart and updates
Reverse Proxy	Nginx (Docker container)	Routes /crypticsync path to the API container, handles port management inside Docker network

3.2 FastAPI vs ASP.NET Core

FastAPI (Python) was evaluated as an alternative. While it produces less boilerplate code, ASP.NET Core was selected for the following reasons:

- Professional production experience with C# allows confident defence of architectural decisions during presentation.
- Dapper's stored procedure support is first-class — `SqlMapper.QueryAsync` with `DynamicParameters` maps directly to MySQL SP parameters.
- JWT middleware in ASP.NET Core is mature, well-documented, and integrates cleanly with role-based `[Authorize]` attributes.
- Visual Studio provides best-in-class debugging, profiling, and test tooling for C# projects.
- ASP.NET Core Web API carries stronger enterprise credibility in an assignment context.

3.3 Hosting Architecture

The application uses a split hosting strategy — the frontend is hosted on a cloud CDN while the backend runs locally on a developer's PC, exposed securely to the internet via Cloudflare Tunnel:

- crypticsync.rakshitr.co.in — Angular SPA deployed to Vercel. Auto-deploys on every push to the main branch on GitHub. SSL is provisioned automatically.
- api.rakshitr.co.in/crypticsync — ASP.NET Core API running inside Docker on the local PC. Cloudflare Tunnel creates an encrypted outbound connection from the local machine to Cloudflare's edge without requiring port forwarding or a static IP. SSL is handled by Cloudflare.
- MySQL runs in a Docker container on the same Docker Compose network as the API, accessible only internally — never exposed to the public internet.

4. Feature List

4.1 What They Asked For

Authentication & Access Control

- Secure login for every employee using Employee ID + Password
- Role-based access: Employee, Manager, HR/SuperUser
- JWT session management with token expiry
- Each user sees only data they are authorized to view

Individual Employee Dashboard

- View personal attendance for any date range (from Date of Joining)
- Display First IN and Last OUT per day
- Show 1st Half and 2nd Half status (PR, AB, WO, PH, FB, RD, Leave Codes)
- Show Gross Work Hours and Net Work Hours per day
- Monthly summary view: present days, absent days, half-days, weekly offs, public holidays

Manager Dashboard (Hierarchical)

- Manager can view their own attendance plus all direct reports' attendance
- If a reportee is also a manager, their sub-team attendance is visible too (recursive hierarchy)
- Team attendance summary for selected date or month
- Drill down from team summary to individual employee detail

Data Source Integration

- Pull employee master from Matrix COSEC User API (/api.svc/v2/user)
- Pull daily records from COSEC Daily Attendance API (/api.svc/v2/attendance-daily)
- Pull monthly summaries from COSEC Monthly Attendance API (/api.svc/v2/attendance-monthly)
- XLS upload fallback when API is unavailable (see Section 12)

4.2 What We Give Extra (Value-Adds)

- Punch timeline view showing all biometric events per day (up to 12 punches from COSEC PUNCH1–PUNCH12 fields)
- Late In / Early Out visual indicators per attendance record
- Shift information displayed per day (Shift Start, Shift End, Shift Type)
- Summary remarks per day (e.g., Absent due to late-in — from COSEC SUMMARY field)
- Day Status badge clearly distinguishing WO, PH, WO+PH, and Normal working days
- Colour-coded team attendance table for managers (green = present, red = absent, orange = half-day)
- Department-wise filter for managers with large teams
- Export team attendance to Excel or PDF for a selected period
- HR Admin panel: sync attendance on demand, manage users, manage hierarchy, import employees from XLS
- Swagger UI available at /crypticsync/swagger for API testing (disabled in production)

4.3 Future Scope

- Leave Management: apply, approve/reject, track balances (Paid, Casual, Sick, Unpaid)
- Email/push notification alerts for absent employees, unapproved leaves, low work hours
- Month-on-month attendance trend charts and department-level absenteeism heatmaps
- Overtime accumulation tracker using COSEC AUTHORIZEDOVERTIME and MANOTCR fields
- Scheduled auto-sync with COSEC API via background cron service
- Payroll integration hook — attendance data feeds salary computation
- Watched folder service — automatically process XLS files dropped into a designated folder
- Progressive Web App (PWA) for mobile browser experience
- SSO / Active Directory integration
- Localization (regional languages) and ADA accessibility compliance
- Month-end attendance lock (no edits after payroll cutoff date)
- Regularization request flow — employee flags an incorrect record for HR review
- Full audit log of all HR and admin actions

5. Diagrams

5.1 Use Case Diagram

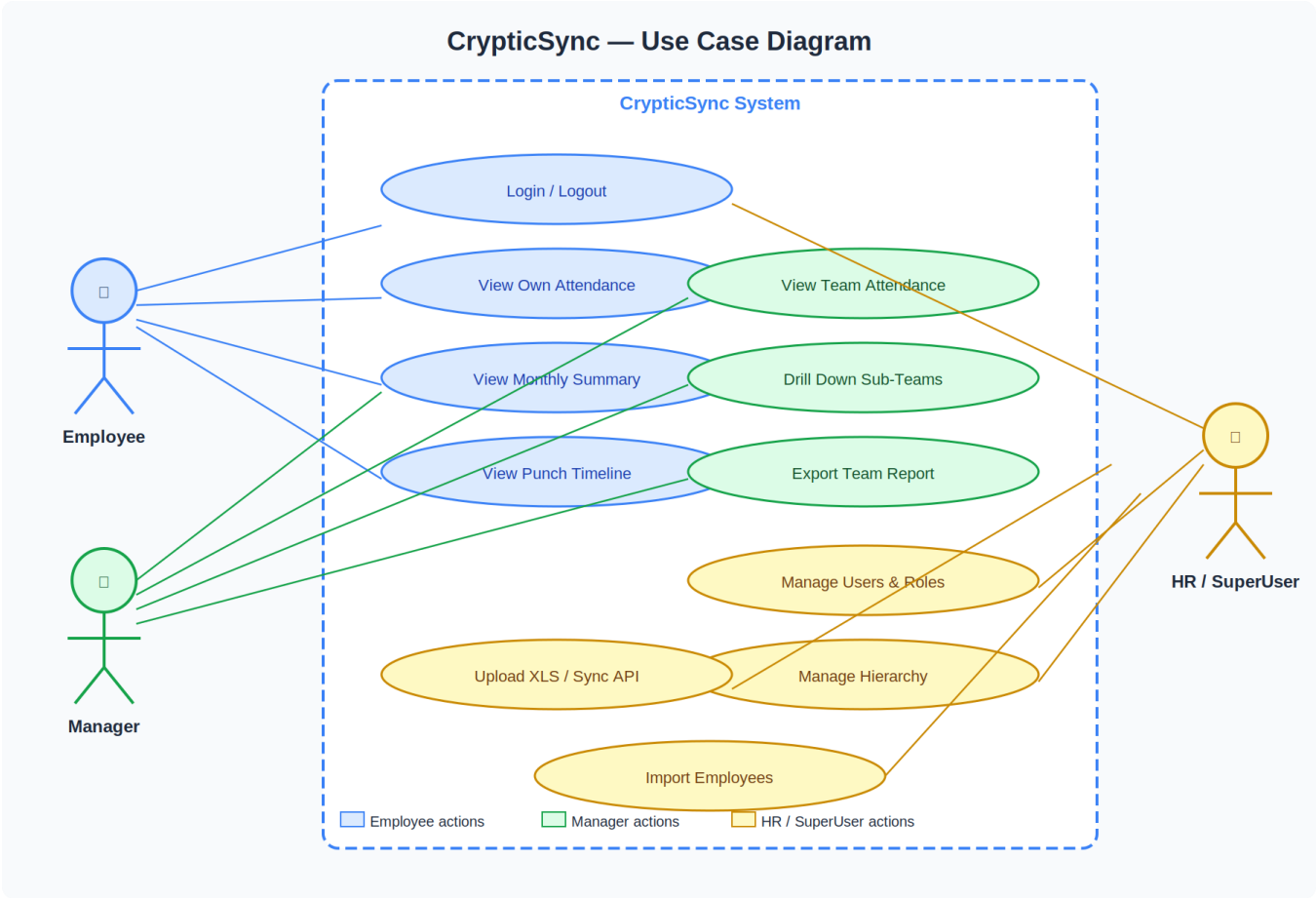


Figure 1 — Use Case Diagram showing actions available to each role

5.2 System Architecture Diagram

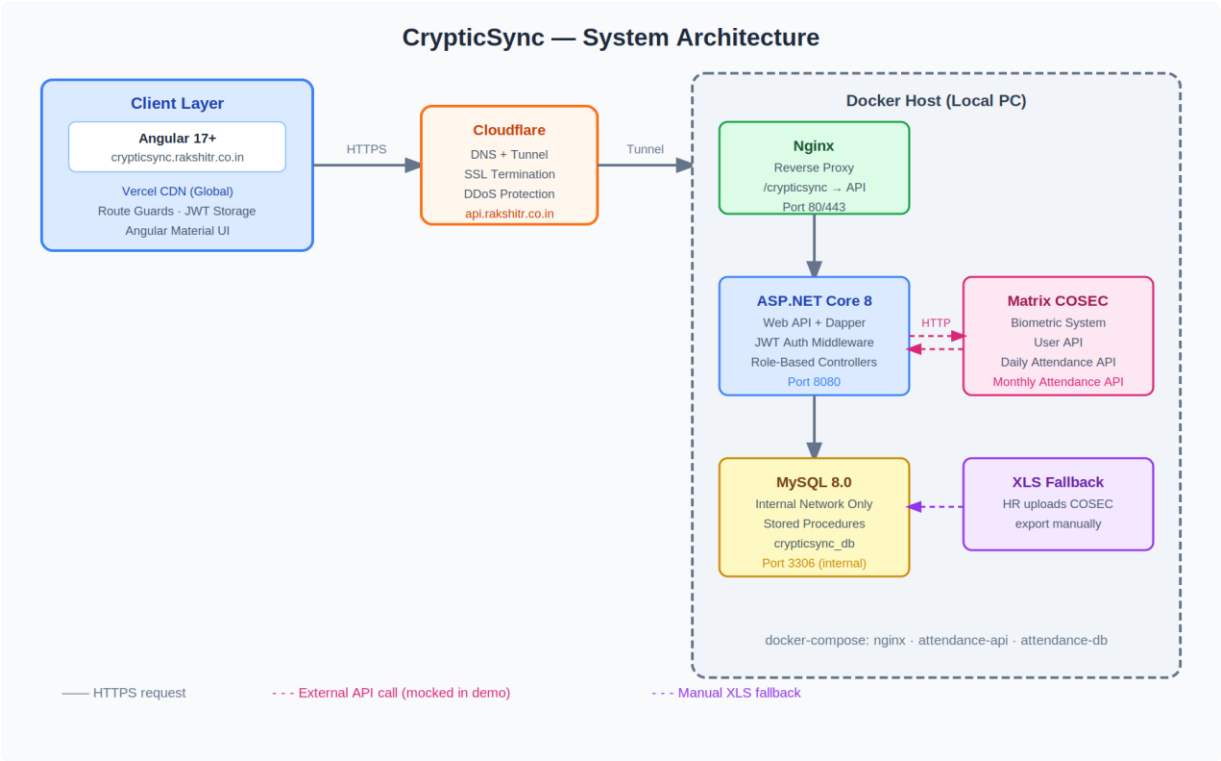


Figure 2 — System Architecture: Angular (Vercel) → Cloudflare → Docker Host (Nginx, API, MySQL) ↔ COSEC

5.3 Data Flow Diagram

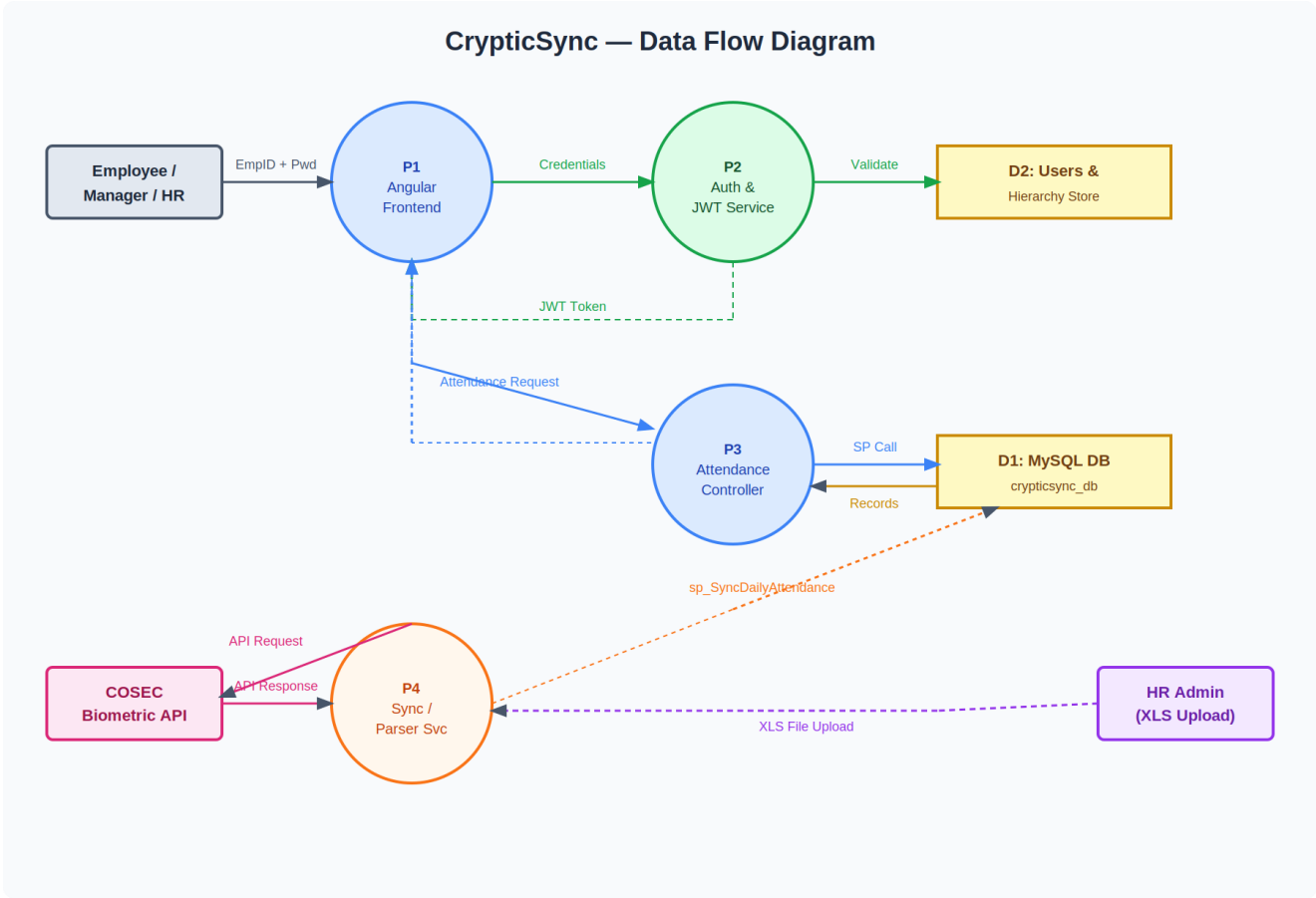


Figure 3 — Data Flow Diagram showing how data moves between actors, processes, and data stores

5.4 Sequence Diagram

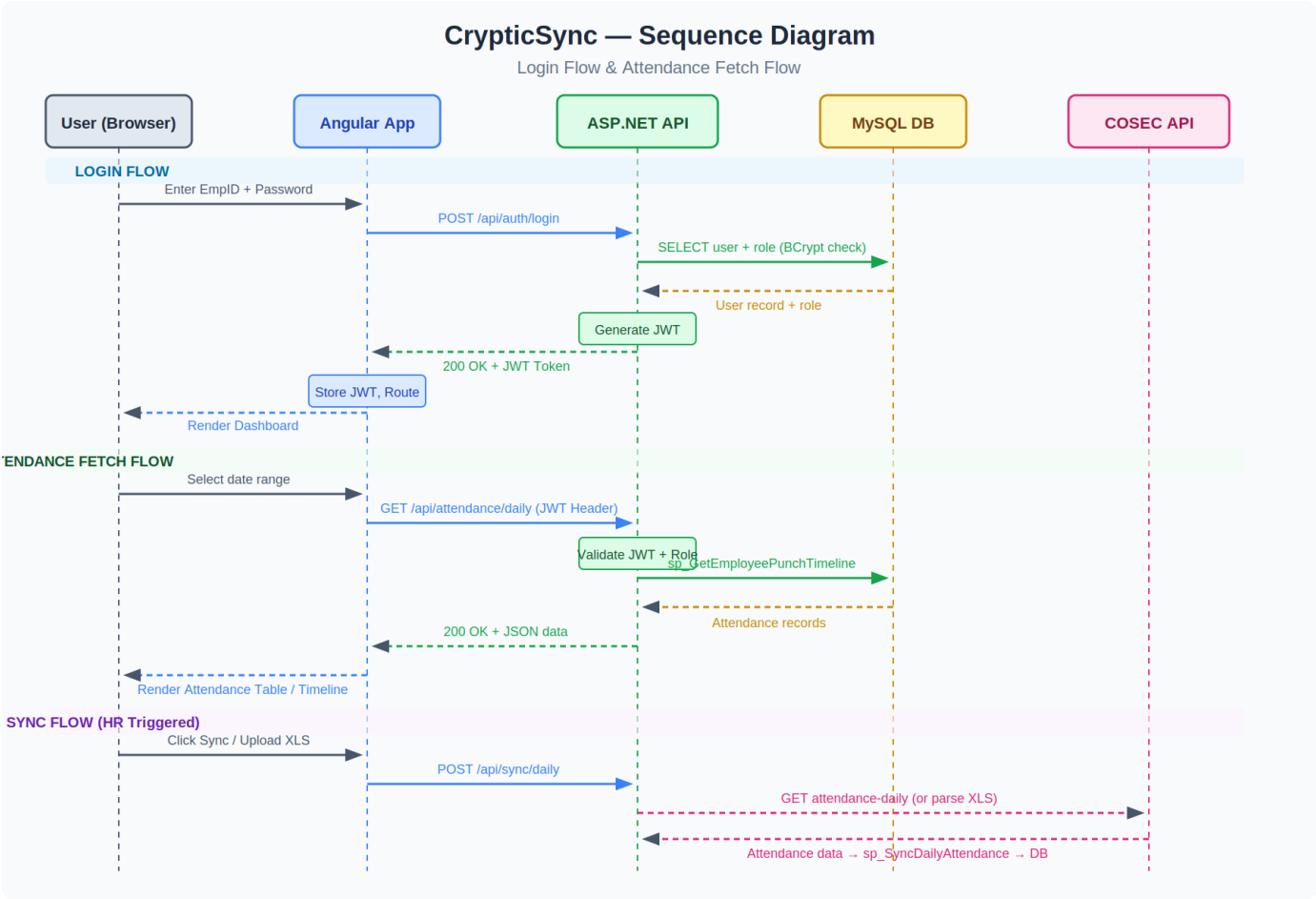


Figure 4 — Sequence Diagram: Login Flow, Attendance Fetch Flow, and COSEC Sync Flow

5.5 Entity Relationship Diagram

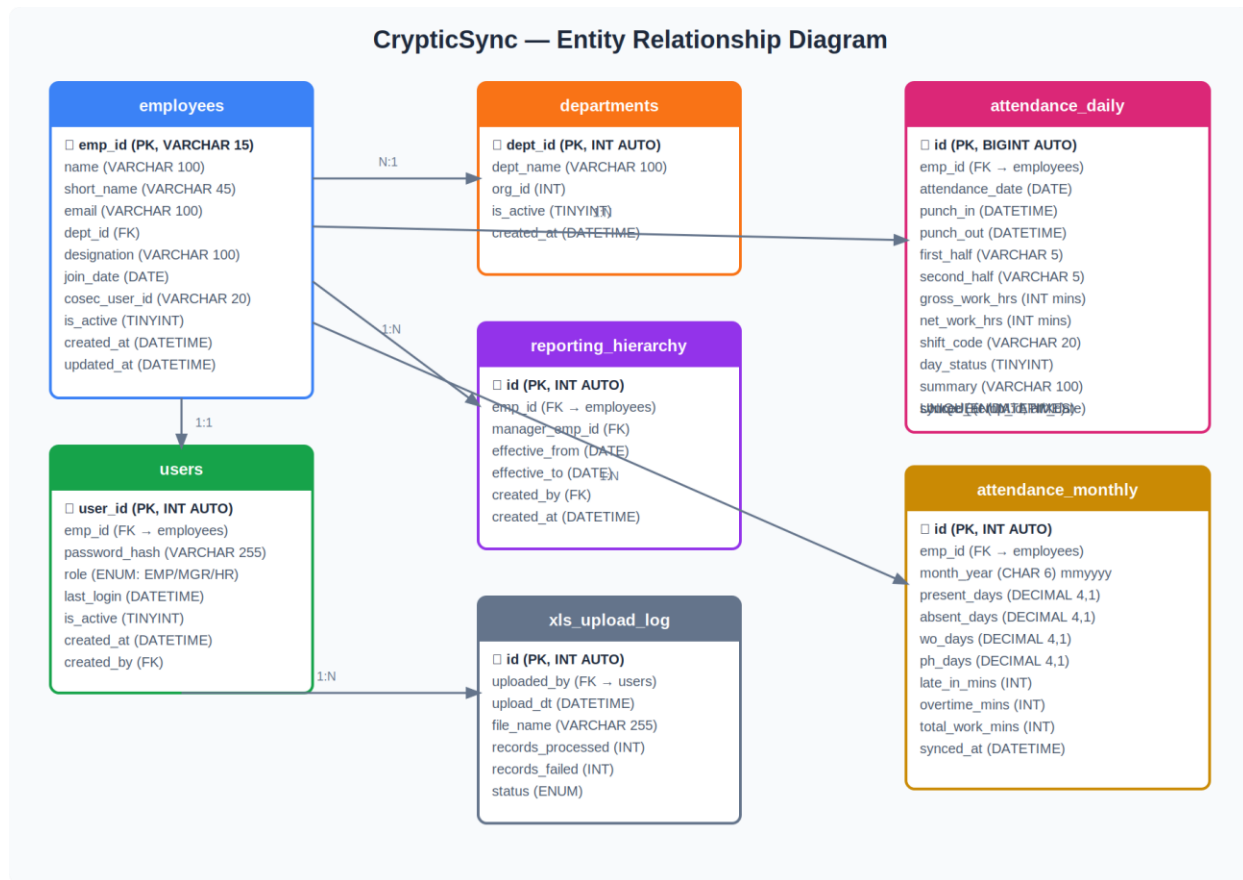


Figure 5 — ER Diagram showing all tables, columns, and relationships in crypticsync_db

5.6 Deployment Diagram

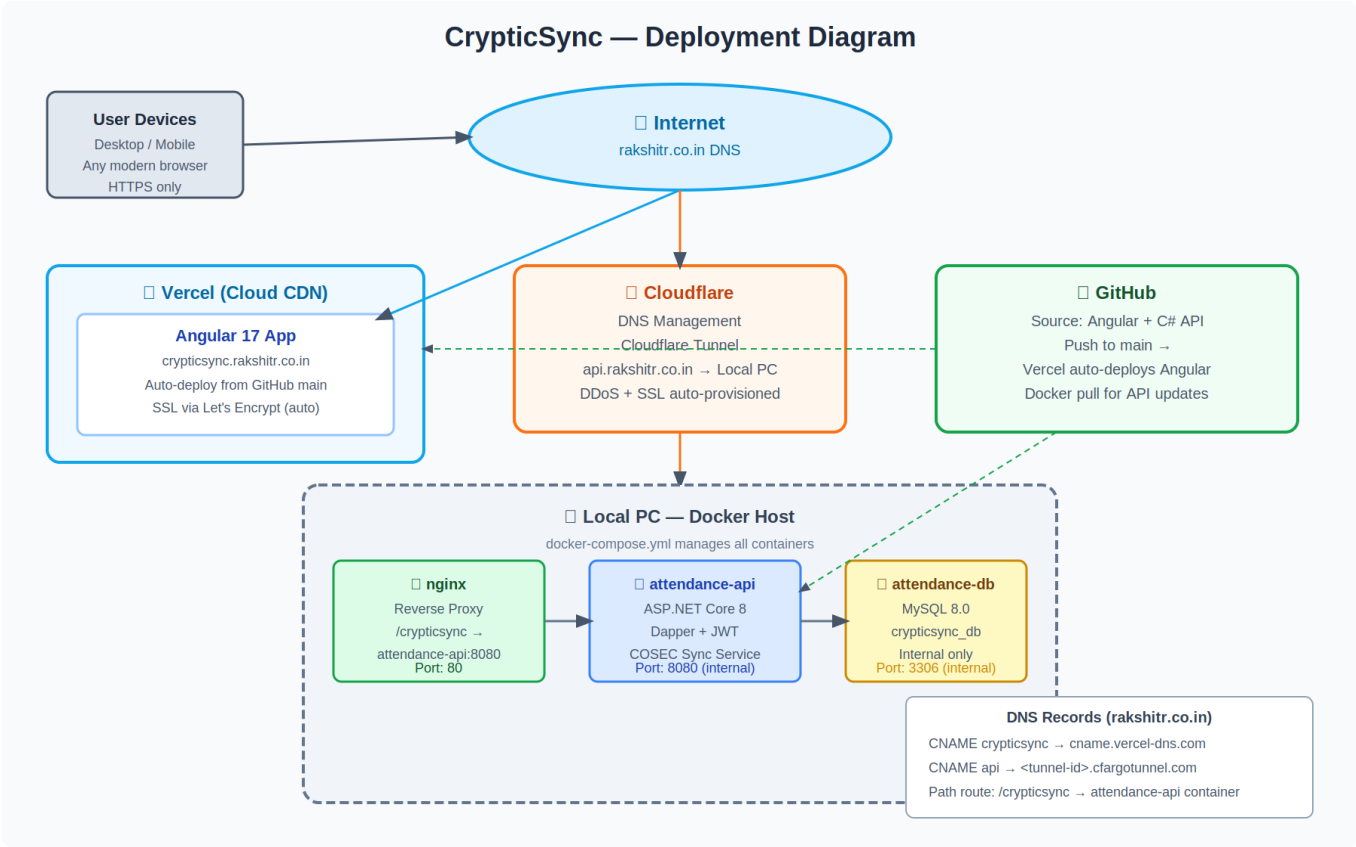


Figure 6 — Deployment Diagram: GitHub → Vercel (frontend) and Docker Host via Cloudflare Tunnel (backend)

6. Database Schema

The MySQL database is named crypticsync_db. The schema is designed to separate concerns cleanly: employee master data, login credentials, hierarchy, daily attendance cache, monthly attendance cache, and upload audit logs are all in separate tables.

6.1 Table: employees

Column	Type	Notes
emp_id	VARCHAR(15) PK	Matches COSEC USERID field
name	VARCHAR(100)	Full name from COSEC USERNAME
short_name	VARCHAR(45)	From COSEC short_name
email	VARCHAR(100)	Optional, used for notifications in Phase 2
dept_id	INT FK	References departments.dept_id
designation	VARCHAR(100)	Job title
join_date	DATE	Attendance history synced from this date
cosec_user_id	VARCHAR(20)	ADLUSERID from COSEC — for API queries
is_active	TINYINT(1)	0 = inactive, 1 = active
created_at	DATETIME	Record creation timestamp
updated_at	DATETIME	Last update timestamp

6.2 Table: users

Column	Type	Notes
user_id	INT AUTO PK	Internal user ID
emp_id	VARCHAR(15) FK	References employees.emp_id (1:1)
password_hash	VARCHAR(255)	BCrypt hashed password
role	ENUM	EMPLOYEE, MANAGER, HR
last_login	DATETIME	Timestamp of most recent login
is_active	TINYINT(1)	HR can deactivate accounts
created_at	DATETIME	Account creation timestamp
created_by	INT FK	users.user_id of HR who created the account

6.3 Table: departments

Column	Type	Notes
dept_id	INT AUTO PK	Internal department ID
dept_name	VARCHAR(100)	Department name (from COSEC DPTID mapping)
org_id	INT	Organisation ID from COSEC ORGID
is_active	TINYINT(1)	Soft delete flag
created_at	DATETIME	Record creation timestamp

6.4 Table: reporting_hierarchy

Column	Type	Notes
id	INT AUTO PK	Internal ID
emp_id	VARCHAR(15) FK	The subordinate employee
manager_emp_id	VARCHAR(15) FK	The manager this employee reports to
effective_from	DATE	Start of this reporting relationship
effective_to	DATE NULL	NULL = currently active
created_by	INT FK	HR user who created this record
created_at	DATETIME	Record creation timestamp

6.5 Table: attendance_daily

Column	Type	Notes
id	BIGINT AUTO PK	Internal ID
emp_id	VARCHAR(15) FK	References employees.emp_id
attendance_date	DATE	The date of this attendance record
punch_in	DATETIME NULL	First biometric punch (PUNCH1 from COSEC)
punch_out	DATETIME NULL	Last biometric punch (OUTPUNCH from COSEC)
first_half	VARCHAR(5)	PR/AB/WO/PH/FB/RD/IN or leave code
second_half	VARCHAR(5)	PR/AB/WO/PH/FB/RD/IN or leave code
gross_work_mins	INT	Gross work duration in minutes
net_work_mins	INT	Net work hours (N-Punch) in minutes
shift_code	VARCHAR(20)	Shift identifier (e.g., GS)

Column	Type	Notes
day_status	TINYINT	0=WO, 1=PH, 2=WO+PH, 3=Normal
summary	VARCHAR(100) NULL	COSEC SUMMARY field — human readable note
source	ENUM	API or XLS — indicates data origin
synced_at	DATETIME	When this record was last synced
UNIQUE KEY	emp_id + attendance_date	Prevents duplicate records

6.6 Table: attendance_monthly

Column	Type	Notes
id	INT AUTO PK	Internal ID
emp_id	VARCHAR(15) FK	References employees.emp_id
month_year	CHAR(6)	Format: mmyyyy (e.g., 042026 for April 2026)
present_days	DECIMAL(4,1)	Full and half days present (multiples of 0.5)
absent_days	DECIMAL(4,1)	Absent days
wo_days	DECIMAL(4,1)	Weekly off days
ph_days	DECIMAL(4,1)	Public holiday days
late_in_mins	INT	Total late arrival minutes for the month
early_out_mins	INT	Total early departure minutes for the month
overtime_mins	INT	Total authorized overtime minutes
total_work_mins	INT	Total net work minutes for the month
synced_at	DATETIME	When this record was last synced
UNIQUE KEY	emp_id + month_year	One row per employee per month

6.7 Table: xls_upload_log

Column	Type	Notes
id	INT AUTO PK	Internal ID
uploaded_by	INT FK	users.user_id — must be HR role
upload_dt	DATETIME	When the file was uploaded
file_name	VARCHAR(255)	Original file name for audit trail
date_range_from	DATE	Earliest attendance date in the file
date_range_to	DATE	Latest attendance date in the file
records_found	INT	Total rows parsed from the file
records_imported	INT	Rows successfully written to DB
records_failed	INT	Rows skipped due to validation errors
status	ENUM	PENDING, PROCESSING, COMPLETED, FAILED
error_log	TEXT NULL	JSON array of row-level error messages

7. Stored Procedures

All database interactions from the ASP.NET Core API go through stored procedures. This encapsulates business logic in the database layer, provides a clean contract between application and DB, and allows performance tuning independently of application code.

7.1 sp_SyncDailyAttendance

Property	Detail
Purpose	Upsert a single daily attendance record fetched from COSEC API or parsed from XLS. Uses INSERT ... ON DUPLICATE KEY UPDATE to handle re-syncs safely.
Called By	SyncService (C#) after fetching COSEC daily API response or parsing XLS upload
Parameters	p_emp_id VARCHAR(15), p_att_date DATE, p_punch_in DATETIME, p_punch_out DATETIME, p_first_half VARCHAR(5), p_second_half VARCHAR(5), p_gross_mins INT, p_net_mins INT, p_shift VARCHAR(20), p_day_status TINYINT, p_summary VARCHAR(100), p_source ENUM
Returns	Rows affected (0 = no change, 1 = inserted, 2 = updated)
Key Logic	UNIQUE constraint on (emp_id, attendance_date) prevents duplicates. ON DUPLICATE KEY UPDATE refreshes all mutable columns and updates synced_at timestamp.

7.2 sp_GetTeamAttendanceHierarchy

Property	Detail
Purpose	Recursively fetch all subordinate employee IDs under a given manager, following the reporting hierarchy to any depth. Used to enforce that managers only see their own team data.
Called By	AttendanceController — GET /api/attendance/team endpoint
Parameters	p_manager_emp_id VARCHAR(15) — the logged-in manager's employee ID
Returns	Result set of emp_id values for all direct and indirect reports
Key Logic	Uses a recursive Common Table Expression (WITH RECURSIVE) on the reporting_hierarchy table. Traverses the hierarchy tree until no more children are found. Respects effective_from and effective_to dates — only active relationships are included.

7.3 sp_GetMonthlyAttendanceSummary

Property	Detail
Purpose	Return the monthly attendance summary for one or more employees for a given month. Aggregates present days, absent days, work hours, late minutes, and overtime.
Called By	AttendanceController — GET /api/attendance/monthly endpoint
Parameters	p_emp_id VARCHAR(15) — single employee or NULL for manager bulk fetch; p_month_year CHAR(6) — target month in mmyyyy format
Returns	Result set with emp_id, name, present_days, absent_days, wo_days, ph_days, late_in_mins, overtime_mins, total_work_mins
Key Logic	Reads from attendance_monthly table (pre-aggregated from COSEC monthly API). Falls back to aggregating from attendance_daily if monthly record is missing for a given month.

7.4 sp_GetEmployeePunchTimeline

Property	Detail
Purpose	Return daily attendance records for an employee over a date range, including status fields for the punch timeline view. Verifies the requesting user has access to the requested employee's data.
Called By	AttendanceController — GET /api/attendance/daily endpoint
Parameters	p_emp_id VARCHAR(15) — target employee; p_date_from DATE; p_date_to DATE; p_requesting_user_id INT — used for access validation
Returns	Result set with attendance_date, punch_in, punch_out, first_half, second_half, gross_work_mins, net_work_mins, shift_code, day_status, summary, source for each day in range
Key Logic	Validates that p_requesting_user_id either owns the emp_id, or is a manager in the hierarchy above p_emp_id (calls sp_GetTeamAttendanceHierarchy result set for the check). Returns empty set — not an error — if access is denied, preventing data leakage through error messages.

8. Angular Components & Pages

The Angular application is structured with feature modules for each role, a shared module for reusable components, and a core module for auth guards and HTTP interceptors.

8.1 Application Structure

Module / Component	Route	Access	Purpose
AppModule	/	Public	Root module, routing setup, HTTP interceptor registration
AuthModule > LoginComponent	/login	Public	EmpID + Password login form. On success, stores JWT and routes by role.
EmployeeModule > DashboardComponent	/employee/dashboard	Employee+	Personal attendance summary cards — present days, absent, work hours this month
EmployeeModule > AttendanceCalendarComponent	/employee/attendance	Employee+	Month calendar view with colour-coded day status. Click day to see punch timeline.
EmployeeModule > PunchTimelineComponent	/employee/punch/:date	Employee+	Shows all punch events for a day, IN/OUT times, shift info, and summary remark
EmployeeModule > ProfileComponent	/employee/profile	Employee+	View personal details: name, department, designation, joining date
ManagerModule > TeamDashboardComponent	/manager/team	Manager+	Team attendance overview table — all direct and indirect reports for selected date/month
ManagerModule > HierarchyViewComponent	/manager/hierarchy	Manager+	Visual org-tree of manager's reporting chain with attendance status indicators
ManagerModule > TeamExportComponent	/manager/export	Manager+	Date range picker and export trigger for Excel/PDF team report

Module / Component	Route	Access	Purpose
HRModule > AdminDashboardComponent	/hr/dashboard	HR only	System-level stats: total employees, sync status, recent XLS uploads
HRModule > UserManagementComponent	/hr/users	HR only	Create, edit, activate/deactivate user accounts. Assign roles.
HRModule > HierarchyManagementComponent	/hr/hierarchy	HR only	Define and edit reporting relationships between employees
HRModule > SyncManagementComponent	/hr/sync	HR only	Trigger on-demand COSEC API sync for daily or monthly data
HRModule > XLSUploadComponent	/hr/upload	HR only	XLS drag-and-drop upload, parse preview table, flagged rows review, import confirmation
SharedModule > NavbarComponent	All routes	All	Top navigation with role-based menu items and logout
SharedModule > SidebarComponent	All routes	All	Collapsible side menu for navigation within the current role module
SharedModule > StatusBadgeComponent	Inline	All	Colour-coded badge for PR/AB/WO/PH/FB/RD status codes
SharedModule > DateRangePickerComponent	Inline	All	Reusable date range selector used across attendance views
SharedModule > LoadingSkeletonComponent	Inline	All	Skeleton placeholder shown while API calls are in flight
CoreModule > AuthGuard	Route Guard	All	Redirects unauthenticated users to /login
CoreModule > RoleGuard	Route Guard	All	Checks JWT role claim — redirects to appropriate dashboard if

Module / Component	Route	Access	Purpose
			accessing wrong module
CoreModule > JwtInterceptor	HTTP Interceptor	All	Automatically attaches Authorization: Bearer <token> header to all API requests

9. API Endpoints (ASP.NET Core)

Base URL: <https://api.rakshitr.co.in/crypticsync/api>. All endpoints except /auth/login require a valid JWT Bearer token. Role requirements are enforced via [Authorize(Roles = "...")] attributes.

9.1 Authentication

Method	Endpoint	Access	Description
POST	/auth/login	Public	Accepts { empld, password }. Returns JWT token + role + employee details on success.
POST	/auth/logout	All roles	Client-side token discard. Endpoint exists for future token blacklist implementation.

9.2 Attendance

Method	Endpoint	Access	Description
GET	/attendance/daily?empld=&from=&to=	All roles	Daily punch records for an employee. Validates access via sp_GetEmployeePunchTimeLine.
GET	/attendance/monthly?empld=&monthYear=	All roles	Monthly summary for an employee. Returns aggregated data from attendance_monthly.
GET	/attendance/team?managerId=&from=&to=	Manager, HR	Team attendance for all subordinates of the given manager for a date range.
GET	/attendance/team/summary?managerId=&monthYear=	Manager, HR	Monthly summary for entire team — used for the colour-coded team dashboard.

9.3 Employees

Method	Endpoint	Access	Description
GET	/employees	HR	List all employees with active flag and department.
GET	/employees/{empld}	HR, Manager	Get single employee details.
POST	/employees	HR	Create a new employee record (from manual entry or XLS import).

Method	Endpoint	Access	Description
PUT	/employees/{empld}	HR	Update employee details (designation, dept, join date).
PATCH	/employees/{empld}/deactivate	HR	Soft-delete an employee — sets is_active = 0.

9.4 Users & Hierarchy

Method	Endpoint	Access	Description
GET	/users	HR	List all user accounts with roles.
POST	/users	HR	Create login account for an employee. Hashes password with BCrypt.
PUT	/users/{id}	HR	Update role or reset password.
PATCH	/users/{id}/deactivate	HR	Deactivate login account.
GET	/hierarchy/{managerId}	Manager, HR	Get all direct reports of a manager.
GET	/hierarchy/{managerId}/full	Manager, HR	Get recursive full team tree.
POST	/hierarchy	HR	Create a new reporting relationship.
DELETE	/hierarchy/{id}	HR	Remove a reporting relationship (sets effective_to = today).

9.5 Sync & Upload

Method	Endpoint	Access	Description
POST	/sync/daily	HR	Trigger on-demand sync from COSEC Daily API for a given date range and employee set.
POST	/sync/monthly	HR	Trigger on-demand sync from COSEC Monthly API for a given month.
POST	/upload/xls	HR	Upload COSEC XLS export. Returns parse preview with flagged rows before import.
POST	/upload/xls/confirm	HR	Confirm and import valid rows from a previously uploaded and parsed XLS file.
GET	/upload/logs	HR	List all XLS upload history with status and error counts.

10. COSEC API Integration Plan

10.1 API Overview

The Matrix COSEC biometric system exposes three REST endpoints used by CrypticSync:

API	Endpoint	Used For
User Data	/api.svc/v2/user?action=get;range=all	Seeding employee master data on initial setup
Daily Attendance	/api.svc/v2/attendance-daily?action=get;date-range=ddmmyyyy-ddmmyyyy;range=all	Fetching per-day punch records for a date range
Monthly Attendance	/api.svc/v2/attendance-monthly?action=get;month-year=mmyyyy;range=all	Fetching pre-aggregated monthly summaries

10.2 Key Field Mapping — Daily Attendance

COSEC Field	Tag Name	CrypticSync Column
User ID	USERID	employees.emp_id
1st Punch	PUNCH1	attendance_daily.punch_in
Out Punch	OUTPUNCH	attendance_daily.punch_out
First Half	FIRSTHALF	attendance_daily.first_half (PR/AB/WO/PH/FB/RD/IN)
Second Half	SECONDHALF	attendance_daily.second_half
Gross Work Hours	NETWORKHRS (minutes)	attendance_daily.gross_work_mins
N-Punch Work Hours	NETWORKHRS	attendance_daily.net_work_mins
Shift Start	SHIFTSTART	Displayed in punch timeline — not stored separately
Day Status	DAYSTATUS	attendance_daily.day_status (0=WO,1=PH,2=WO+PH,3=Normal)
Summary	SUMMARY	attendance_daily.summary
Process Date	PROCESSDATE	attendance_daily.synced_at

10.3 Demo / Mock Strategy

Since the COSEC server is on a private corporate network, a mock COSEC adapter is implemented for the assignment demonstration. The ICosecApiClient interface is implemented by two classes:

- CosecApiClient — real implementation that calls the COSEC HTTP endpoints. Used in production.

- **MockCosecApiClient** — returns static JSON data matching the COSEC API response schema. Registered in the DI container when `COSEC MOCK=true` environment variable is set.

This approach satisfies the Dependency Inversion Principle and allows switching between real and mocked COSEC data with a single environment variable change, requiring zero code changes.

11. Security Approach

Concern	Approach
Authentication	JWT Bearer tokens issued on login. Token contains emp_id, user_id, and role claim. Signed with HS256 using a secret key stored in environment variables.
Token Expiry	8-hour expiry on JWT tokens. Users must re-login after expiry. Refresh token support is planned for Phase 2.
Password Storage	BCrypt with cost factor 12. Plain-text passwords are never stored or logged anywhere in the system.
Role Enforcement (API)	[Authorize(Roles = "EMPLOYEE,MANAGER,HR")] attributes on all controllers. Role hierarchy is additive — HR can access all Manager and Employee endpoints.
Role Enforcement (Angular)	AuthGuard and RoleGuard on all routes. Angular route configuration prevents navigation to unauthorized pages without a network request.
Data Isolation	sp_GetEmployeePunchTimeline validates that the requesting user is either the emp_id owner or is a manager in their hierarchy. Unauthorized requests return empty result sets, not errors.
HTTPS	All traffic is encrypted. Vercel auto-provisions SSL for the frontend. Cloudflare Tunnel provides SSL termination for the API.
CORS	The ASP.NET Core API is configured to accept requests only from https://crypticsync.rakshitr.co.in in production. Local development allows localhost:4200.
SQL Injection	All DB calls use parameterized queries through Dapper. No raw SQL string concatenation anywhere in the codebase.
Secrets Management	No secrets in source code. JWT signing key, MySQL credentials, and COSEC API credentials are stored in Docker environment variables and .env files excluded from version control.
Swagger	Swagger UI is enabled only when ASPNETCORE_ENVIRONMENT=Development. It is disabled in the production Docker container.

12. Alternative Approach — When API is Unavailable

12.1 Why the API May Be Unavailable

- The COSEC server is typically on a private corporate LAN — not internet-accessible by default.
- Corporate IT firewall policies may block external API access to the biometric system.
- The COSEC system may be undergoing maintenance or upgrades.
- In smaller deployments, the COSEC server may not have API licensing enabled.

12.2 The Fallback: XLS Upload Pipeline

The Matrix COSEC software already supports exporting attendance data as a formatted XLS file — which is exactly what the provided sample file represents. CrypticSync implements an XLS Upload page (accessible to HR/SuperUser only) that accepts this export file and processes it into the same MySQL database tables, via the same stored procedures, as the live API sync. The end-user experience for employees and managers is identical regardless of whether data arrived via API or XLS.

12.3 XLS Upload Flow

Step	What Happens
1. Export	HR exports the attendance XLS from the COSEC software (existing functionality — no change to COSEC setup required)
2. Upload	HR navigates to CrypticSync Admin > Upload XLS and drags the file into the upload zone
3. Parse	The ASP.NET Core API parses the XLS using EPPlus (C# library). It reads the company header, detects employee sections by the ID-Name pattern, and maps column data to the attendance schema.
4. Preview	A preview table is returned to the Angular frontend showing: employee name, date, First IN, Last OUT, status, work hours, and a flag icon for any rows with issues (unknown employee, missing punch, unrecognized shift code).
5. Review	HR reviews flagged rows. A summary bar shows: X records found, Y employees detected, Z rows flagged for review.
6. Confirm	HR clicks Import Valid Records. The API calls <code>sp_SyncDailyAttendance</code> for each valid row using <code>INSERT ... ON DUPLICATE KEY UPDATE</code> — safe to re-import the same file without duplication.
7. Log	The upload is recorded in <code>xls_upload_log</code> with file name, date range, record counts, and any error details. This creates an audit trail of all manual imports.

12.4 Alternatives Considered and Rejected

Alternative	Why Rejected
Watched Folder (drop XLS into a server folder)	Requires persistent filesystem access inside Docker — breaks clean container isolation. Also less user-friendly than a UI upload.
Email Attachment Parser	Requires mailbox credentials, IMAP setup, and significant additional infrastructure. Overkill for the problem.
Manual DB Entry Form	Someone would have to type every punch record by hand — defeats the purpose of having a biometric system.
CSV/Mock Data Import	Does not use real COSEC data. Does not answer the assignment question about a genuine fallback.
Screen Scraping COSEC UI	COSEC has no public web UI to scrape. Brittle, maintenance-heavy, and potentially a terms-of-service violation.

13. Future Scope

Phase 2 — Leave Management

- Employee can apply for leave (Sick, Casual, Earned, Unpaid) from within CrypticSync
- Manager receives notification and can approve or reject with comments
- Leave balance tracker using COSEC PLDAYS and ULDDAYS from the Monthly API
- Leave calendar overlay on the attendance calendar view

Phase 2 — Notifications & Alerts

- Email alerts when an employee has not clocked in by a configured threshold time
- Manager notification for team members with consecutive absences
- Low net work hours alert at week-end
- Leave approval / rejection notification to employee

Phase 2 — Advanced Sync

- Scheduled auto-sync via ASP.NET Core IHostedService background worker — runs nightly to sync previous day's COSEC data
- Watched folder service — monitors a configured directory for new XLS files and processes them automatically
- Incremental sync — only fetch records newer than the last synced timestamp

Phase 3 — Analytics

- Month-on-month attendance trend charts per employee
- Department-level absenteeism heatmap
- Late arrival frequency distribution chart
- Overtime accumulation tracker across the team

Phase 3 — Integrations & Accessibility

- Payroll integration hook — attendance data feeds salary computation system
- SSO / Active Directory login for enterprise deployment
- Regularization request flow — employee flags an incorrect attendance record for HR review and correction
- Month-end attendance lock — prevent changes after payroll cutoff date
- Full audit log of all HR and admin actions with before/after values
- Multi-language support (localization) starting with Kannada and Hindi
- ADA / WCAG 2.1 AA accessibility compliance — screen reader support, keyboard navigation, sufficient contrast ratios
- Progressive Web App (PWA) packaging for mobile browser installation
- Geo-fencing for location-based check-in verification

Appendix A — Docker Compose Structure

The following represents the logical structure of the docker-compose.yml that manages all backend services on the local PC:

```
services:
  nginx:
    image: nginx:alpine
    ports: ["80:80"]
    volumes:
      - ./nginx.conf:/etc/nginx/nginx.conf
    depends_on: [attendance-api]
  attendance-api:
    build: ./CrypticSync.API
    environment:
      - MYSQL_HOST=attendance-db
      - JWT_SECRET=${JWT_SECRET}
      - COSEC MOCK=true
    depends_on: [attendance-db]
  attendance-db:
    image: mysql:8.0
    environment:
      - MYSQL_DATABASE=crypticsync_db
      - MYSQL_ROOT_PASSWORD=${DB_ROOT_PWD}
    volumes:
      - db_data:/var/lib/mysql
  db_data:
    volumes:
      - db_data:
```

Appendix B — DNS Configuration

Record Type	Name	Value	Purpose
CNAME	crypticsync	cname.vercel-dns.com	Routes crypticsync.rakshitr.co.in to Vercel
CNAME	api	<tunnel-id>.cfargotunnel.com	Routes api.rakshitr.co.in through Cloudflare Tunnel to local PC

Appendix C — COSEC Status Code Reference

Code	Meaning	Displayed As
PR	Present	Green badge — Present
AB	Absent	Red badge — Absent
WO	Weekly Off	Grey badge — Week Off
PH	Public Holiday	Blue badge — Holiday
FB	Field Break	Purple badge — Field Break
RD	Rest Day	Grey badge — Rest Day
IN	Official In (on duty outside office)	Teal badge — Official Duty
Leave Code	Company-configured leave type (e.g., SL, CL, EL)	Orange badge — On Leave